SRU	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS & AI) Batch: 2025 – 2029
	Name: P. VINNU Reg. No: 2503A52023
	Department: Computer Science and AI Semester: I
	AI Assisted Coding – Assignment: 12.3 Date: 03-09-2026

Q1 – AI-Assisted Bubble Sort Analysis

Question:

- Given numbers = [64, 34, 25, 12, 22, 11, 90], implement Bubble Sort in ascending order.
- Count the number of comparisons and swaps made while sorting.
- Test the algorithm on at least three input lists and compare an early-exit optimised version again original.

Prompt (used with the AI assistant):

"Explain one complete pass of Bubble Sort on [64, 34, 25, 12, 22, 11, 90], then show me how to add an ea optimisation that stops the algorithm once no swaps occur in a pass."

Code:

```
def bubble_sort(arr):
    arr = arr.copy()
    n = len(arr)
    comparisons = 0
    swaps = 0
    for i in range(n - 1):
        for j in range(n - 1 - i):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
    return arr, comparisons, swaps

def bubble_sort_optimised(arr):
    arr = arr.copy()
    n = len(arr)
    comparisons = 0
    swaps = 0
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
                swapped = True
        if not swapped:
            break  # early-exit optimisation
    return arr, comparisons, swaps

test_lists = [
    [64, 34, 25, 12, 22, 11, 90],
    [1, 2, 3, 4, 5],
    [5, 4, 3, 2, 1],
]
```

```
for lst in test_lists:
    sorted_arr, cmp, sw = bubble_sort(lst)
    print(f"Input: {lst}")
    print(f"  Original:  {sorted_arr} comparisons={cmp}, swaps={sw}")
    sorted_arr2, cmp2, sw2 = bubble_sort_optimised(lst)
    print(f"  Optimised: {sorted_arr2} comparisons={cmp2}, swaps={sw2}")
```

Output:

```
Input: [64, 34, 25, 12, 22, 11, 90]
  Original:  [11, 12, 22, 25, 34, 64, 90]  comparisons=21, swaps=14
  Optimised: [11, 12, 22, 25, 34, 64, 90]  comparisons=21, swaps=14
Input: [1, 2, 3, 4, 5]
  Original:  [1, 2, 3, 4, 5]  comparisons=10, swaps=0
  Optimised: [1, 2, 3, 4, 5]  comparisons=4, swaps=0
Input: [5, 4, 3, 2, 1]
  Original:  [1, 2, 3, 4, 5]  comparisons=10, swaps=10
  Optimised: [1, 2, 3, 4, 5]  comparisons=10, swaps=10
PS C:\Users\thann\OneDrive\Desktop\AI-Assisted  PROJECT>
```

Observation:

- Both versions sort every list correctly and produce the same final output.
- On the already-sorted list [1,2,3,4,5], the optimised version needed only 4 comparisons and made
  instead of the original's 10 comparisons across 4 passes — the early-exit check (swapped flag) de
  sorted array immediately.
- On the reverse-sorted list [5,4,3,2,1], every pass still performs at least one swap, so the optimised
  versions perform identically (10 comparisons, 10 swaps) — the early exit only helps when the da
  sorted before the last pass.
- Time complexity remains $O(n^2)$ in the worst case for both versions; the optimisation improves th
  $O(n)$ for already-sorted input.

Q2 – Linear Search vs Binary Search

Question:

- Given numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89] and target = 45, implement Linear Search and Binary Search.
- Count the number of comparisons made by each algorithm.
- Test both algorithms for the first, middle, last, and an absent element.

Prompt (used with the AI assistant):

*"Explain why Binary Search requires the input array to be sorted, and suggest a way to reduce compari
Search for an already-sorted array."*

Code:

```
def linear_search(arr, target):
    comparisons = 0
    for i, val in enumerate(arr):
        comparisons += 1
        if val == target:
            return i, comparisons
    return -1, comparisons

def binary_search(arr, target):
    comparisons = 0
    low, high = 0, len(arr) - 1
    while low <= high:
```

```
    mid = (low + high) // 2
    comparisons += 1
    if arr[mid] == target:
      return mid, comparisons
    elif arr[mid] < target:
      low = mid + 1
    else:
      high = mid - 1
  return -1, comparisons

numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
targets = {first (5)': 5, 'middle (31)': 31, 'last (89)': 89,
       'absent (100)': 100, 'assignment target (45)': 45}

for label, t in targets.items():
  li, lc = linear_search(numbers, t)
  bi, bc = binary_search(numbers, t)
  print(f'{label}: Linear -> index {li}, cmp {lc} | Binary -> index {bi}, cmp {bc}')
```

Output:

```
first (5): Linear -> index 0, cmp 1 | Binary -> index 0, cmp 3
middle (31): Linear -> index 4, cmp 5 | Binary -> index 4, cmp 1
last (89): Linear -> index 8, cmp 9 | Binary -> index 8, cmp 4
absent (100): Linear -> index -1, cmp 9 | Binary -> index -1, cmp 4
assignment target (45): Linear -> index 5, cmp 6 | Binary -> index 5, cmp 3
```

Observation:

- Linear Search performs well only when the target is near the start (1 comparison for the first ele
degrades to O(n) for the last element or an absent element (9 comparisons on a 9-element list).
- Binary Search needs at most log2(9) = 4 comparisons for any case, including the absent elemen
eliminates half the search space at every step.
- Binary Search requires sorted data because its decision to move left or right depends on compari
middle element against the target — this only correctly narrows the range when the array is ordered.
- For small or unsorted, frequently-changing data, Linear Search is simpler and avoids the cost of
large, static, sorted datasets, Binary Search is clearly superior (O(log n) vs O(n)).

Q3 – Compare Sorting Algorithms

Question:

- Given data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56], sort it using Bubble Sort, Selection Sort, and Insertion Sort.
- Use the same input for all three algorithms and count comparisons and swaps/shifts.
- Compare the experimental results with the theoretical time complexity of each algorithm.

Prompt (used with the AI assistant):

*"Compare Bubble Sort, Selection Sort, and Insertion Sort on this dataset — explain their strengths, weak
which one is best suited for nearly-sorted data."*

Code:

```
def bubble_sort(arr):
  arr = arr.copy(); n = len(arr); cmp = 0; sw = 0
  for i in range(n - 1):
    for j in range(n - 1 - i):
      cmp += 1
      if arr[j] > arr[j+1]:
        arr[j], arr[j+1] = arr[j+1], arr[j]; sw += 1
  return arr, cmp, sw

def selection_sort(arr):
```

```
    arr = arr.copy(); n = len(arr); cmp = 0; sw = 0
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            cmp += 1
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]; sw += 1
    return arr, cmp, sw

def insertion_sort(arr):
    arr = arr.copy(); n = len(arr); cmp = 0; sh = 0
    for i in range(1, n):
        key = arr[i]; j = i - 1
        while j >= 0:
            cmp += 1
            if arr[j] > key:
                arr[j+1] = arr[j]; sh += 1; j -= 1
            else:
                break
        arr[j+1] = key
    return arr, cmp, sh

data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
for name, fn in [('Bubble Sort', bubble_sort),
                 ('Selection Sort', selection_sort),
                 ('Insertion Sort', insertion_sort)]:
    sorted_arr, cmp, sw = fn(data)
    print(f'{name}: {sorted_arr} comparisons={cmp}, swaps/shifts={sw}')
```

Output:

```
Bubble Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]  comparisons=45, swaps/shifts=24
Selection Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]  comparisons=45, swaps/shifts=6
Insertion Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]  comparisons=31, swaps/shifts=24
```

Observation:

- All three algorithms correctly produce the sorted list [5, 10, 12, 23, 34, 45, 56, 67, 78, 89].
- Selection Sort made the fewest swaps (6) because it performs exactly one swap per pass, while Bu and Insertion Sort made 24 swaps/shifts each — Selection Sort minimises writes at the cost of alw scanning the full remaining array.
- Insertion Sort made the fewest comparisons (31) versus 45 for both Bubble Sort and Selection So stop scanning early once it finds the correct insertion point.
- All three are $O(n^2)$ in the worst case, matching the theoretical complexity confirmed by the simila comparison counts here (10-element input).
- Insertion Sort is the best choice for nearly-sorted data, since its comparisons drop toward O(n) w elements are out of place — this matches its early-exit inner while loop.

Q4 – Optimising a Student Search and Sort Program

Question:

- Given a list of (name, mark) tuples for 5 students, write functions to find the highest mark, sort st mark, and search for a student by name.
- Start with a straightforward solution, then use AI to identify unnecessary operations and suggest efficient approach.
- Compare the original and improved solutions' time complexity.

Prompt (used with the AI assistant):

"Review this code for finding the highest mark and searching a student by name — are there any unnec

operations, and how can I make these functions more efficient?"

Code:

```
students = [
    ('Arun', 82), ('Bala', 65), ('Chitra', 91),
    ('Divya', 74), ('Esha', 88)
]

# Straightforward (unoptimised) versions
def highest_mark_v1(students):
    sorted_students = sorted(students, key=lambda s: s[1], reverse=True)
    return sorted_students[0]     # O(n log n) just to find one max

def sort_by_mark_v1(students):
    return sorted(students, key=lambda s: s[1])

def search_by_name_v1(students, name):
    for s in students:
        if s[0] == name:
            return s
    return None                   # O(n) linear scan

# AI-suggested improved versions
def highest_mark_v2(students):
    return max(students, key=lambda s: s[1])   # O(n)

def search_by_name_v2(students_dict, name):
    return students_dict.get(name)             # O(1) average

print('Highest (v1):', highest_mark_v1(students))
print('Highest (v2):', highest_mark_v2(students))
print('Sorted by mark:', sort_by_mark_v1(students))
print("Search 'Chitra' (v1):", search_by_name_v1(students, 'Chitra'))
students_dict = dict(students)
print("Search 'Chitra' (v2):", search_by_name_v2(students_dict, 'Chitra'))
print("Search 'Zara' (absent):", search_by_name_v1(students, 'Zara'))
```

Output:

```
Highest (v1): ('Chitra', 91)
Highest (v2): ('Chitra', 91)
Sorted by mark: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]
Search 'Chitra' (v1): ('Chitra', 91)
Search 'Chitra' (v2): 91
Search 'Zara' (absent): None
```

Observation:

- The straightforward highest_mark_v1() sorts the entire list (O(n log n)) just to read off the first el
  suggested highest_mark_v2() uses Python's built-in max() to scan once in O(n) — both return the
  result, ('Chitra', 91), but the improved version does less work.
- search_by_name_v1() scans the list linearly (O(n)) and correctly returns None for an absent nam
  search_by_name_v2() converts the list to a dictionary once and performs an O(1) average-time lo
  more efficient when many repeated searches are needed.
- Sorting by mark (sort_by_mark_v1) is already close to optimal at O(n log n) using Python's Timso
  further improvement was suggested there.
- Overall, the improved solution trades a small one-time cost (building the dictionary) for much fa
  lookups — a good trade-off whenever search_by_name is called more than once.

Q5 – Optimisation Challenge: Best Combination

Question:

- Given a list of items with prices and a budget of ₹50,000, find the combination of items with the h

price that does not exceed the budget.

- Compare Brute Force, Greedy, and Dynamic Programming approaches and determine which gua optimal solution.
- Implement the selected approach and test it with at least three different budgets.

Prompt (used with the AI assistant):

"Compare Brute Force, Greedy, and Dynamic Programming approaches for this 0/1 knapsack-style budg and tell me which one guarantees the optimal answer."

Code:

```
from itertools import combinations

items = [('Laptop', 60000), ('Phone', 30000), ('Tablet', 20000),
         ('Watch', 10000), ('Headphones', 5000)]

def brute_force_best_combo(items, budget):
    best_combo, best_total = [], 0
    for r in range(1, len(items) + 1):
        for combo in combinations(items, r):
            total = sum(p for _, p in combo)
            if total <= budget and total > best_total:
                best_total, best_combo = total, combo
    return best_combo, best_total          # O(2^n)

def dp_best_combo(items, budget):
    n = len(items)
    dp = [[0] * (budget + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        name, price = items[i - 1]
        for b in range(budget + 1):
            dp[i][b] = dp[i - 1][b]
            if price <= b:
                dp[i][b] = max(dp[i][b], dp[i - 1][b - price] + price)
    b, chosen = budget, []
    for i in range(n, 0, -1):
        if dp[i][b] != dp[i - 1][b]:
            name, price = items[i - 1]
            chosen.append((name, price)); b -= price
    return list(reversed(chosen)), dp[n][budget]    # O(n * budget)

for budget in [50000, 35000, 15000]:
    combo_bf, total_bf = brute_force_best_combo(items, budget)
    combo_dp, total_dp = dp_best_combo(items, budget)
    print(f"Budget Rs.{budget}: BruteForce -> {combo_bf} = Rs.{total_bf}")
    print(f"                DP         -> {combo_dp} = Rs.{total_dp}")
```

Output:

```
Budget Rs.50000: BruteForce -> (('Phone', 30000), ('Tablet', 20000)) = Rs.50000
                DP         -> [('Phone', 30000), ('Tablet', 20000)] = Rs.50000
Budget Rs.35000: BruteForce -> (('Phone', 30000), ('Headphones', 5000)) = Rs.35000
                DP         -> [('Phone', 30000), ('Headphones', 5000)] = Rs.35000
Budget Rs.15000: BruteForce -> (('Watch', 10000), ('Headphones', 5000)) = Rs.15000
                DP         -> [('Watch', 10000), ('Headphones', 5000)] = Rs.15000
```

Observation:

- A Greedy approach (e.g., always picking the most expensive item that still fits) is not used here be miss the true optimum — for example, greedily taking the Laptop (₹60,000) first is impossible un ₹50,000 budget, and picking by 'value density' does not guarantee optimality for this 0/1 selection problem.
- Brute Force checks every possible subset ($2^5 = 32$ combinations for 5 items) and is guaranteed op $O(2^n)$ cost becomes impractical as the number of items grows.
- Dynamic Programming (0/1 knapsack) also guarantees the optimal answer, running in O(n × bud

which scales far better for larger item lists or larger budgets.

- For all three tested budgets (₹50,000, ₹35,000, ₹15,000), Brute Force and DP produced identical op combinations and totals, confirming the DP implementation is correct.
- DP is the recommended approach for larger inputs since it avoids the exponential blow-up of Bru while still guaranteeing the optimal combination, at the cost of extra memory for the DP table.